CS4340 – Tutorial 2S (Assessed)

Summary

This tutorial will be assessed, and focuses on the following topic areas:

- Design/Implementation

Please submit your solutions electronically via Blackboard at the end of the tutorial session. Your submission should be a PDF file named after your name and the name of the tutorial i.e., FirstName Surname 2S.pdf.

Q1 – Use Case Diagram

A Use Case diagram allows architects to map out the interactions between users and the 'system' that the architects are planning to implement. Use case diagrams fall between system design, user interface (UX) design and software testing.

For this question, you are required to take the following user interaction description, and convert it into one or more Use Case diagrams, depicting the users' interactions with the proposed system.

You are free to use appropriate UML generation software, but it is suggested that **Draw.io** is used.

Some resources:

https://www.youtube.com/watch?v=nE7gDB2DXCU
https://drawio-app.com/blog/uml-use-case-diagrams-with-draw-io/

Marking Scheme

Diagram (or diagrams) accurately depicting the Technician's interactions with the 'system' **(3 Marks)**
Diagram (or diagrams) accurately depicting the QA Tester's interactions with the 'system' **(3 Marks)**
Each diagram(s) should include boundary, Actor(s) and sufficient Use Case symbols to convey the interactions with the 'system' **(4 marks)**

(10 marks in total)

User Interaction Description for Proposed System

The system is designed to allow the playing of retro arcade/console games on refactored/repurposed Personal Computers (**PC**). In order to reach the end target (i.e. playing games) there are a number of things that the setup technician will need to do, before the eventual customer can successfully play the games on the computer.

The first step is to boot the repurposed PC using USB installation media. Once loaded the technician will select a menu option to install the system onto the repurposed PC's internal Hard Disk Drive (HDD). After rebooting the PC, the technician will be presented with a 'bare bones' system designed to run on the PC (without the USB media being present/needed). Whilst the initial installation (i.e. onto the HDD) is automated, there are some additional manual installation steps required before the next stage (Configuration) can be progressed too. BIOS files need to be installed in the Share\bios folder and the games ('ROMs') will need to be placed into a series of folders (one folder for each emulated arcade/console system) i.e. \Share\roms\vectrex, \Share\roms\gx4000, \Share\roms\sg1000. While not a compulsory requirement, it is usually best practice to access the systems 'Scrape ROMS' menu option which will connect to the Internet, and allow the downloading of images, videos and textual descriptions of each game (for each emulated arcade/console system).

(Interaction Description Continued on Next Page)

With the installation interactions complete, the technician will need to perform some configuration steps, in order to allow the system to successfully play the installed ROMS. For each arcade/console, the system will provide two or more software emulators – by default the system will auto-select the appropriate emulator in order to successfully load the game ROM. However, the best practice to explicitly set/select one of the provided emulators as the default, in order to avoid the auto-select system from selecting a random emulator. Most arcade/console systems use a controller of some type in order to actually play the games/ROMs, and this will need to be configured via a 'Configure Games Controller' menu option (for this system, assume a single Playstation or Xbox 360 style 'gamepad').

The technician will need to test or 'play' some of the installed ROMs, in order to test that the correct emulator is being used/selected and that the ROMs are successfully loaded.

Before the system can be deployed to customers, a QA Tester will 'play test' the system to ensure that the system works (from the customer's point of view). This 'play test' will involve the following steps: The Tester will switch on the repurposed PC to ensure it boots to the system. The provided Controller will be used to navigate the systems main menu, and three emulated systems will be selected for testing. For each emulated system, the tester will browse the game/ROM list (to check that they have been successfully 'scraped'), select + run a game/ROM and then exit the game via pressing the appropriate button on the provided Controller. With the games tested, the Tester will access the 'Themes' Menu, select a Theme to download, select the 'Download' option and then the 'Apply Theme' option (once theme has downloaded). With all of the tests complete, the Tester will select the option to 'Shutdown' the system/PC.

Q2 – Coding Best Practice

One aspect of the implementation and testing process is appropriately formatted and documented code. This in turn, allows future developers to maintain the code, along with enabling QA Testers to test it.

On the next page is a small Java program, that successfully compiles and produces some output*.

However, the original developer clearly believes that the use of formatting and meaningful naming conventions is for scruffy Nerf herders**, and not surprisingly no longer works for the company….

Tasks / Marking Scheme

Your task is to complete the following:

1. Reformat/structure the code (next page) so that it clearly follows the taught convention
   i.e. Clearly indicated Class, Methods and Main program sections of code, Code appropriately indented

   (3 Marks)

2. Informative/sensible variable naming, following a consistent formatting convention. This could be the use of 'Camel Case', for example:

   i.e. for integer variables
   `iterationCount`
   `intIterationCount`, or
   `int_iteration_count`

   Regardless of naming/formatting convention used, the casual reader of the code, should be able to intuitively guess what the variables are, and what they are used for

   (3 Marks)

3. The final reformatted/renamed code will need to be documented, to indicate your understanding of the entire program. This can be achieved in **two alternative** ways

   Inline comments (throughout the final code)

   ```
   // This Method does…..
   // Variables used for calculation
   ```

   You are not required to comment every line of code, but there should be sufficient comments to convey your understanding of the overall purpose of the code.

Alternatively, the use of a **class description / comment block** at the beginning of the program, again conveying your understanding of the overall purpose of the code.

```
/**************************************
* Class A                             *
*                                     *
* This program calculates the total   *
* of the sum (intA + intB) and        *
* outputs to screen                   *
*                                     *
* Methods                             *
* -------                             *
*                                     *
* Method 1...                         *
* Method 2...                         *
*                                     *
**************************************/
```

(4 Marks)

(10 marks in total)

Code

```
import java.util.*;class troutersion1{public static String WibbleToTrout(int wibble){String trout = "";while(wibble > 0){int HalfaWibble = wibble % 2;trout = HalfaWibble + trout;wibble = wibble / 2;}return trout;}public static String WibbleToWombat(int wibble){int HalfaWibble;String Wombat="";char WombatBits[]={'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F'};while(wibble > 0){HalfaWibble = wibble % 16;Wombat = WombatBits[HalfaWibble] + Wombat;wibble = wibble / 16;}return Wombat;}public static String WibbleToMooMoo(int wibble){int HalfaWibble;String MooMoo="";char MooMooBits[]={'0','1','2','3','4','5','6','7'};while(wibble > 0){HalfaWibble = wibble % 8;MooMoo = MooMooBits[HalfaWibble] + MooMoo;wibble = wibble / 8;}return MooMoo;}public static void main(String args[]){System.out.println("The Trout for 10 (Wibble) is: " + WibbleToTrout(10));System.out.println("The Trout for 21 (Wibble) is: " + WibbleToTrout(21));System.out.println("The Trout for 31 (Wibble) is: " + WibbleToTrout(31));System.out.println();System.out.println("The Wombat for 10 (Wibble) is: " + WibbleToWombat(10));System.out.println("The Wombat for 15 (Wibble) is: " + WibbleToWombat(15));System.out.println("The Wombat for 289 (Wibble) is: " + WibbleToWombat(289));System.out.println();System.out.println("The MooMoo for 8 (Wibble) is: " + WibbleToMooMoo(8));System.out.println("The MooMoo for 19 (Wibble) is: " + WibbleToMooMoo(19));System.out.println("The MooMoo for 81 (Wibble) is:: " + WibbleToMooMoo(81));System.out.println();}}
```

* The provided code has been successfully executed in both Visual Studio Code and the W3Schools online (https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler)

** https://starwars.fandom.com/wiki/Nerf_herder